

Python Problem Solving

Description

This repository contains Python solutions for common problem-solving and algorithm-based questions.

The problems are based on real-world scenarios such as attendance analysis, shopping, rainwater collection, employee performance, banking, networking, and appointment scheduling.

Topics Covered

- Arrays
- Strings
- Sliding Window
- Hashing
- Set
- Prefix Sum
- Kadane's Algorithm
- Maximum Product Subarray
- Anagram Grouping
- Consecutive Sequence
- Interval Merging
- Two Pointer Technique

Problems Included

1. Student Attendance Analysis
2. Online Shopping Price Analysis
3. Rainwater Collection System
4. Employee Performance Analysis
5. Product Sales Analysis
6. Customer Purchase History
7. Bank Transaction Analysis
8. Employee Skill Grouping
9. Network Packet Analysis
10. Hospital Appointment Scheduling

Programming Language

- Python

Purpose

The purpose of this repository is to practice problem-solving skills and understand commonly used data structures and algorithms in Python.

output:

Programiz

Python Online Compiler

Programiz PRO

main.py

Visualize

Run

Output

```
1 def longest_unique(arr):
2     seen = set()
3     left = 0
4     maximum = 0
5
6     for right in range(len(arr)):
7         while arr[right] in seen:
8             seen.remove(arr[left])
9             left += 1
10
11        seen.add(arr[right])
12        maximum = max(maximum, right - left + 1)
13
14    return maximum
15
16 arr = list(map(int, input().split()))
17 print(longest_unique(arr))
```

```
1 2 3 1 4 5
5
```

main.py

Visualize

Run

Output

```
1 def max_sum(arr):
2     current = maximum = arr[0]
3
4     for i in range(1, len(arr)):
5         current = max(arr[i], current + arr[i])
6         maximum = max(maximum, current)
7
8     return maximum
9
10
11 arr = list(map(int, input().split()))
12 print(max_sum(arr))
```

```
-2 3 4 -1 5
11
```

main.py

Visualize

Run

Output

```
1 def trap(height):
2     left = 0
3     right = len(height) - 1
4     left_max = 0
5     right_max = 0
6     water = 0
7
8     while left < right:
9         if height[left] < height[right]:
10             if height[left] >= left_max:
11                 left_max = height[left]
12             else:
13                 water += left_max - height[left]
14             left += 1
15         else:
16             if height[right] >= right_max:
17                 right_max = height[right]
18             else:
19                 water += right_max - height[right]
20             right -= 1
21
22    return water
23
24
25 height = list(map(int, input().split()))
26 print(trap(height))
```

```
0 1 0 2 1 0 1 3
5
```

LEARN MORE

main.py

VisualizeRun ▶

```
1 def max_performance(arr):
2     current = maximum = arr[0]
3
4     for i in range(1, len(arr)):
5         current = max(arr[i], current + arr[i])
6         maximum = max(maximum, current)
7
8     return maximum
9
10
11 arr = list(map(int, input().split()))
12 print(max_performance(arr))
```

Output

```
-2 5 -1 6 -3
10
```

main.py

VisualizeRun ▶

```
1 def max_product(arr):
2     maximum = minimum = result = arr[0]
3
4     for i in range(1, len(arr)):
5         if arr[i] < 0:
6             maximum, minimum = minimum, maximum
7
8         maximum = max(arr[i], maximum * arr[i])
9         minimum = min(arr[i], minimum * arr[i])
10
11     result = max(result, maximum)
12
13     return result
14
15
16 arr = list(map(int, input().split()))
17 print(max_product(arr))
```

Output

```
2 3 -2 4
6
```

main.py

VisualizeRun ▶

```
1 def longest_unique(arr):
2     seen = set()
3     left = 0
4     maximum = 0
5
6     for right in range(len(arr)):
7         while arr[right] in seen:
8             seen.remove(arr[left])
9             left += 1
10
11         seen.add(arr[right])
12         maximum = max(maximum, right - left + 1)
13
14     return maximum
15
16
17 arr = list(map(int, input().split()))
18 print(longest_unique(arr))
```

Output

```
10 20 30 20 40 50
4
```

main.py

VisualizeRun ▶

Output

```
1 def count_subarrays(arr, target):
2     prefix_sum = 0
3     count = 0
4     frequency = {0: 1}
5
6     for value in arr:
7         prefix_sum += value
8
9         if prefix_sum - target in frequency:
10             count += frequency[prefix_sum - target]
11
12         frequency[prefix_sum] = frequency.get(prefix_sum, 0) + 1
13
14     return count
15
16
17 arr = list(map(int, input().split()))
18 target = int(input())
19
20 print(count_subarrays(arr, target))
```

1 2 3 2
5
2

Programiz

Python Online Compiler

main.py

VisualizeRun ▶

Output

```
1 def group_skills(arr):
2     groups = {}
3
4     for word in arr:
5         key = ''.join(sorted(word))
6         groups.setdefault(key, []).append(word)
7
8     return list(groups.values())
9
10
11 arr = input().split()
12 print(group_skills(arr))
```

eat tea ate bat tab
[['eat', 'tea', 'ate'], ['bat', 'tab']]

Python

main.py

Visualize

Run ▶

1 def longest_consecutive(arr):

2 numbers = set(arr)

3 maximum = 0

4

5 for num in numbers:

6 if num - 1 not in numbers:

7 current = num

8 length = 1

9

10 while current + 1 in numbers:

11 current += 1

12 length += 1

13

14 maximum = max(maximum, length)

15

16 return maximum

17

18

19 arr = list(map(int, input().split()))

20 print(longest_consecutive(arr))

Output

100 4 200 1 3 2

4

Python Online Compiler

main.py

Visualize

Run ▶

1 def merge_intervals(intervals):

2 intervals.sort()

3 result = []

4

5 for start, end in intervals:

6 if not result or start > result[-1][1]:

7 result.append([start, end])

8 else:

9 result[-1][1] = max(result[-1][1], end)

10

11 return result

12

13

14 n = int(input())

15 intervals = []

16

17 for _ in range(n):

18 start, end = map(int, input().split())

19 intervals.append([start, end])

20

21 print(merge_intervals(intervals))

Output

4

1 3

2 6

8 10

9 12

[[1, 6], [8, 12]]

Conclusion

These programs provide simple implementations of commonly used algorithms and techniques for solving programming problems.